

Advanced Programming Techniques

fbeit

FACHBEREICH ELEKTROTECHNIK
UND INFORMATIONSTECHNIK

Prof. Dr. Michael Lipp

fbeit

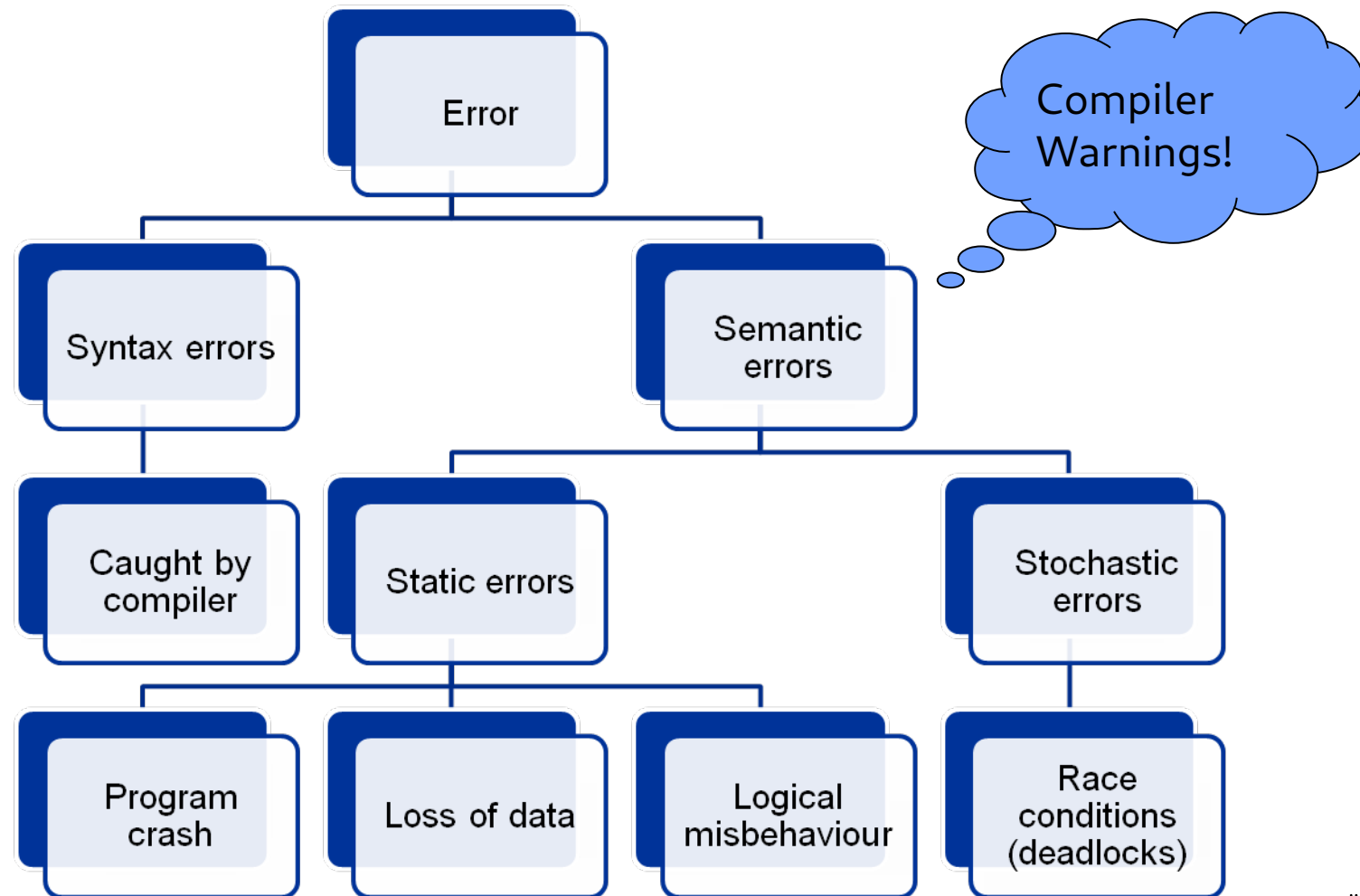
FACHBEREICH ELEKTROTECHNIK
UND INFORMATIONSTECHNIK

Error Handling

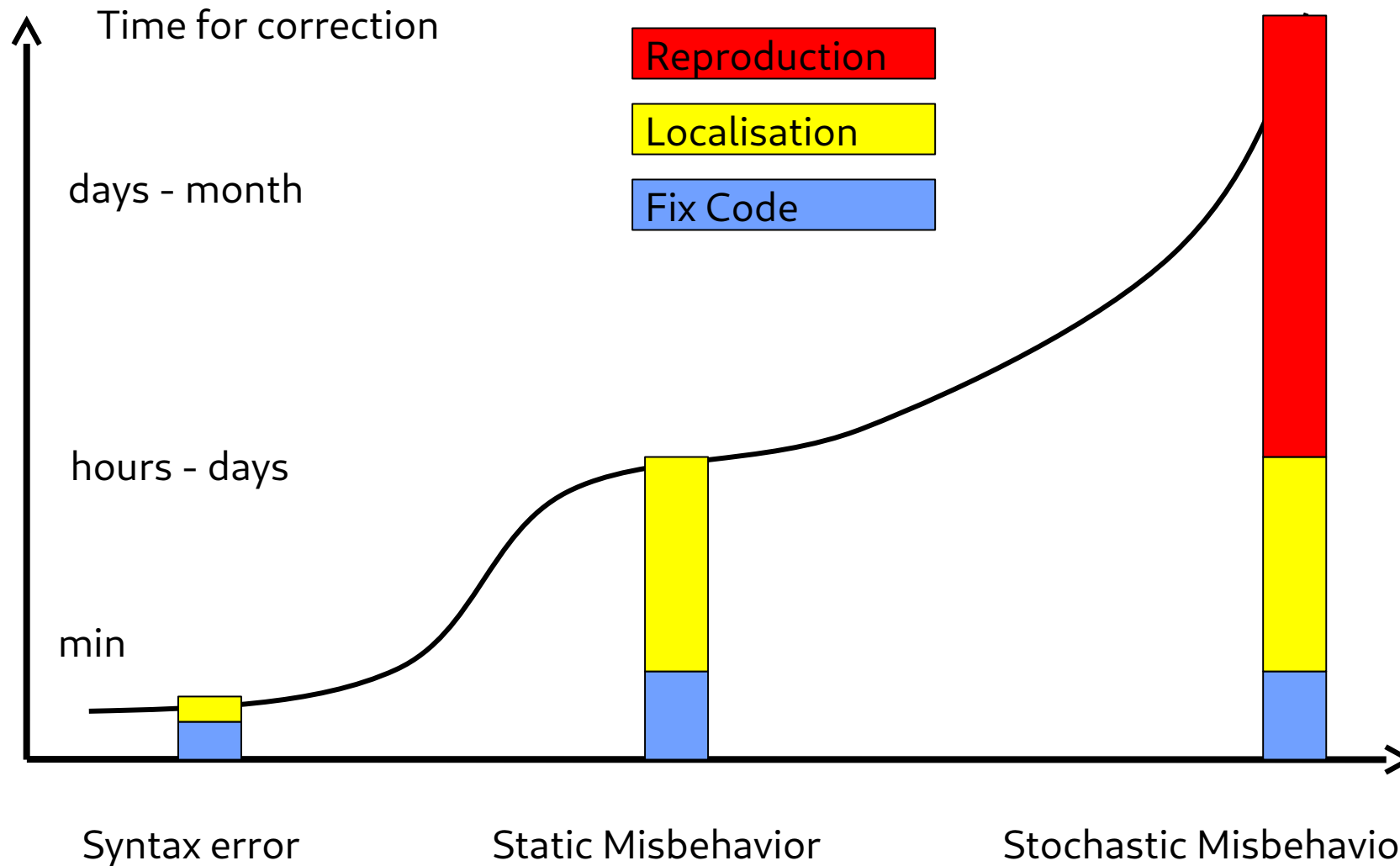
Software Errors

- A software bug is an error, flaw, mistake, failure, fault or “undocumented feature” in a computer program that prevents it from behaving as intended (e.g., producing an incorrect or unexpected result).
 - Most bugs arise from mistakes and errors made by people in either a program's source code or its design, and a few are caused by compilers producing incorrect code. A program that contains a large number of bugs, and/or bugs that seriously interfere with its functionality, is said to be buggy. Reports detailing bugs in a program are commonly known as bug reports, fault reports, problem reports, trouble reports, change requests, and so forth.

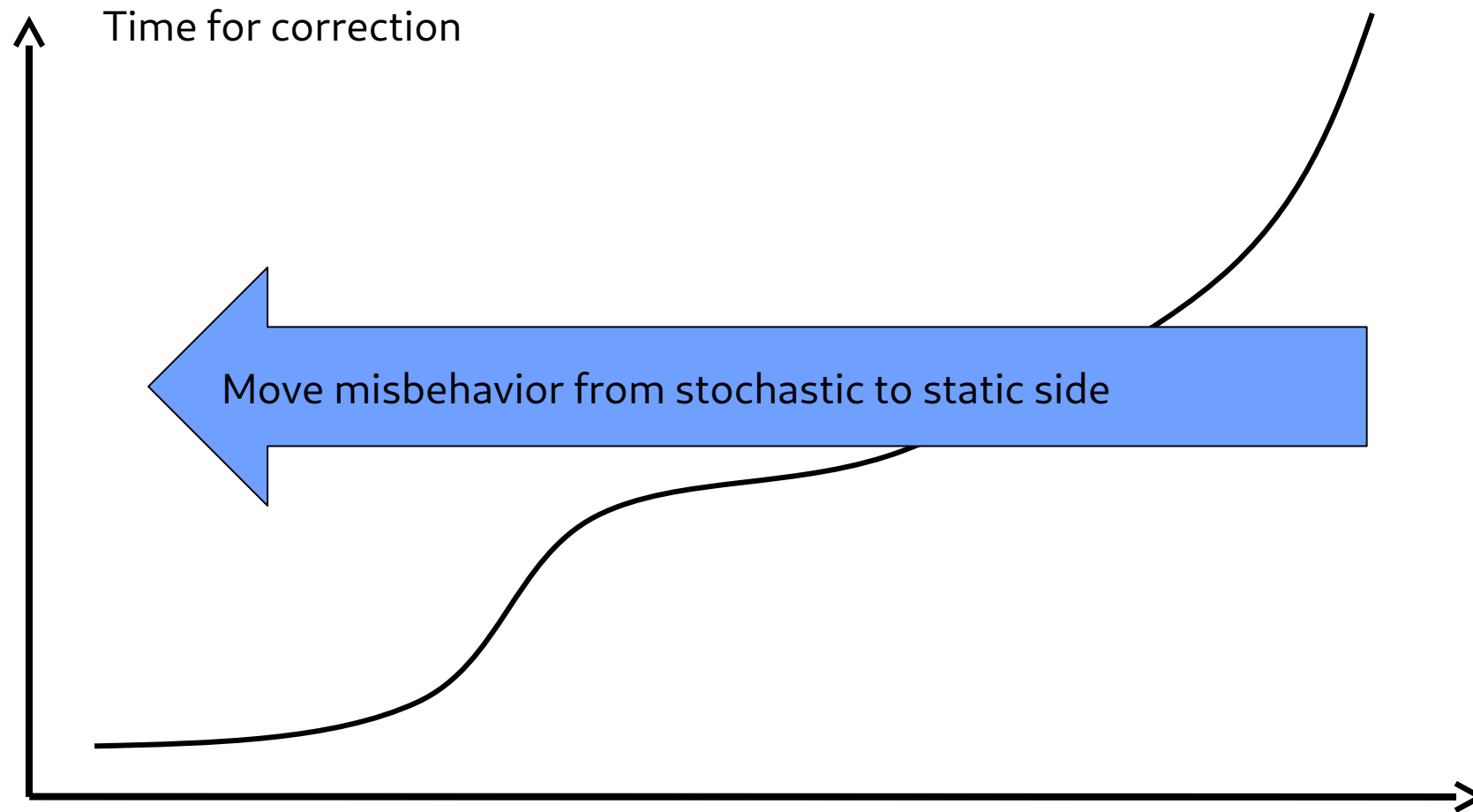
Error Classification



Error Correction Costs



Defensive Programming



Syntax error

Static Misbehavior

Stochastic Misbehavior

Typical Causes for Runtime Bugs

Data

- Processing of wrong data (e.g. division by zero)
- Working with uninitialized data
- Wrong type casting resulting in a loss of data
- Off-by-One (for (i=0; i<= a; i++))
- Assignment versus equality check (if a=b)

Pointers

- Working with the address instead of the content
- Working with deallocated memory

Illegal configuration

- Typical problem in a component based development
- Wrong configuration of a reused component
- E.g. configuration says 3 CAN controllers, but microcontroller only has 2

Static

Typical Causes for Runtime Bugs

State

- Unexpected state change
- Forgotten transitions

Multitasking

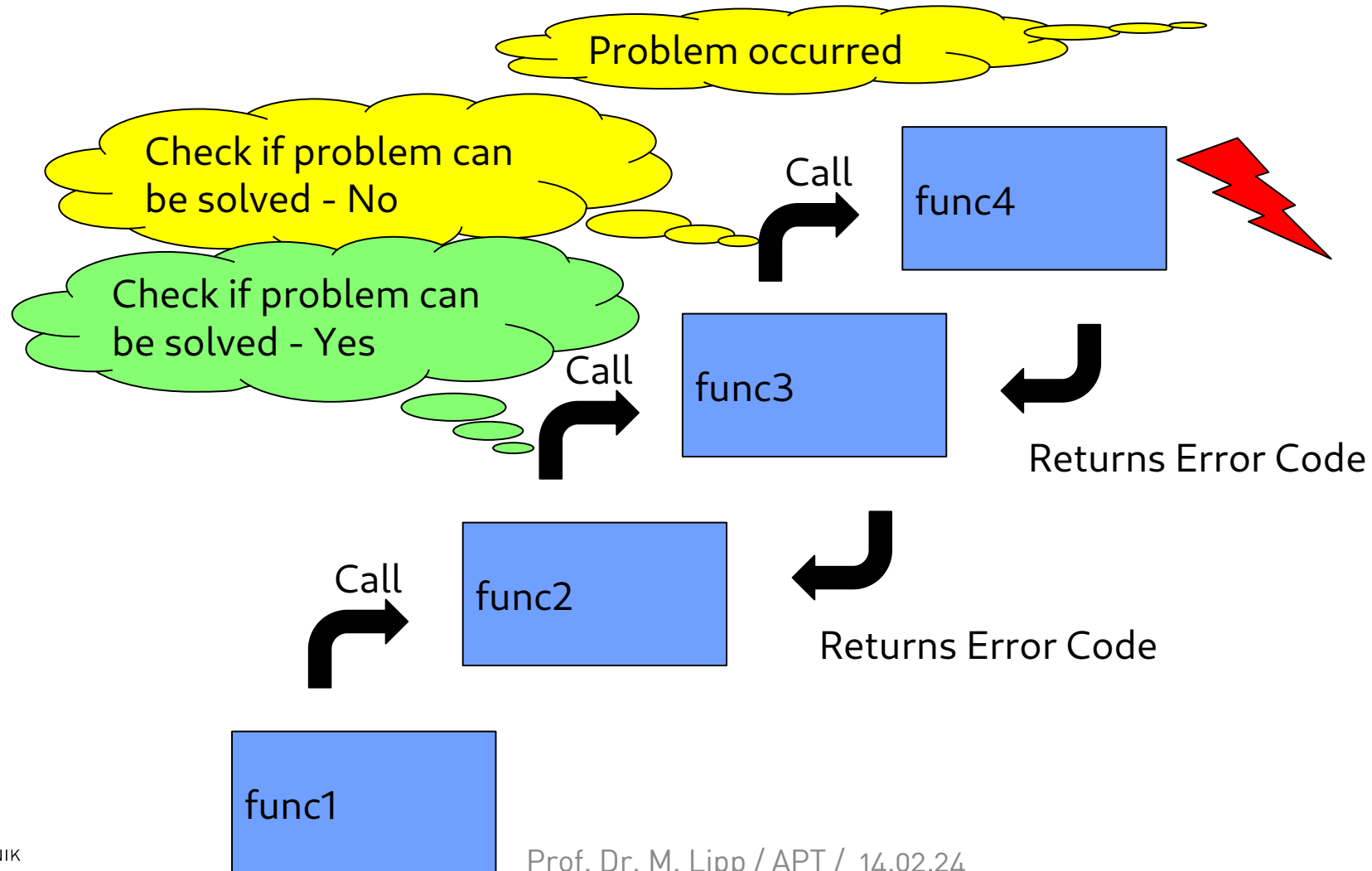
- Unprotected access to shared data
- Forgotten semaphore protection for critical sections
- Deadlocks

Realtime

- Violated realtime constraints

Stochastic

Hierarchical Error Handling



Hierarchical Error Handling

Advantage:

- Problem can be solved on the lowest possible level
- Principle of subsidiarity
- Easy to understand

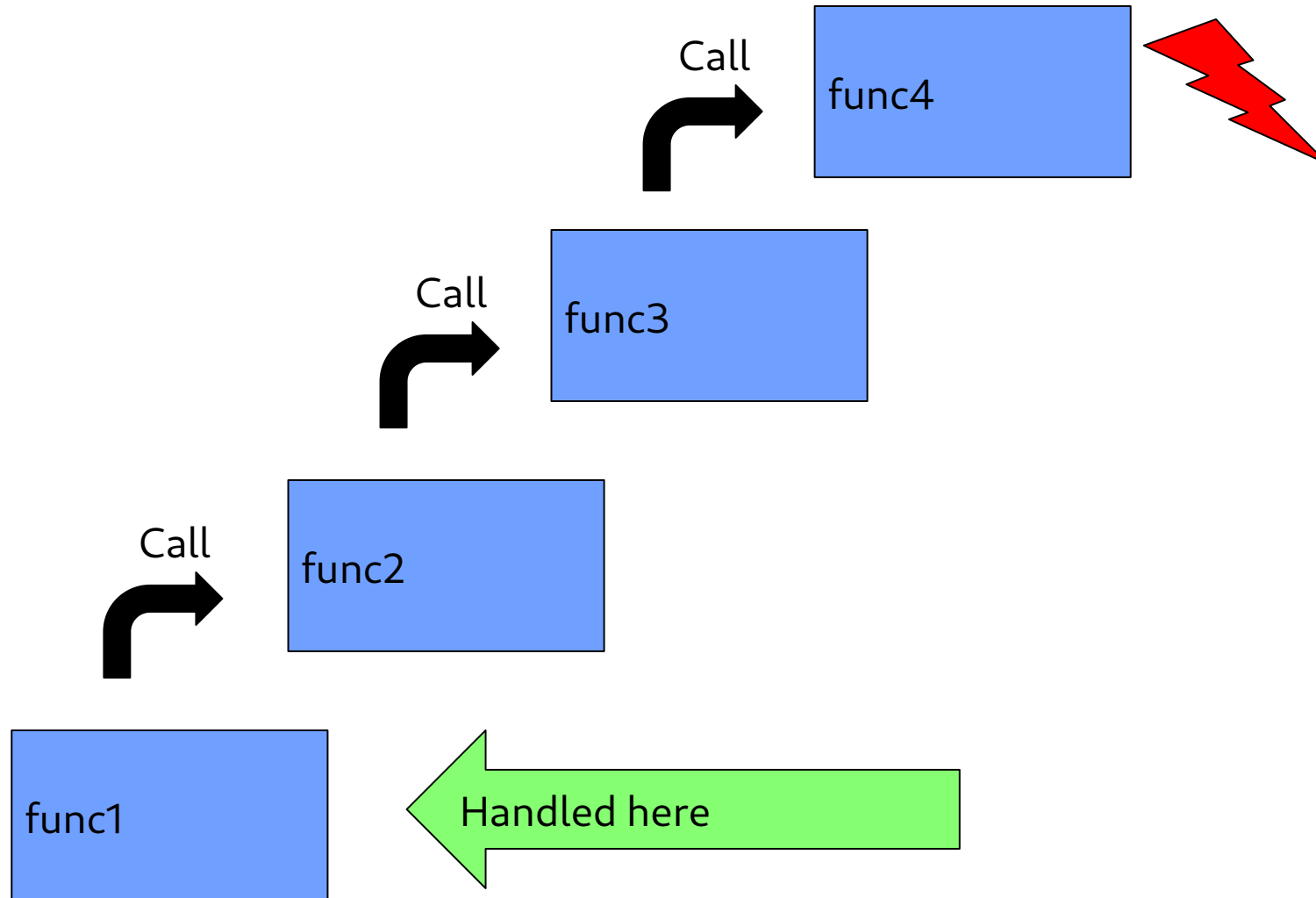
Disadvantage:

- Complex return value handling, as every call level adds own status information
- Mixture of program logic and exception handling may lead to a reduction of maintainability and performance
- Difficult to use in a big team (unclear responsibilities)

Exception Handling

- The general idea of C++ exception handling is to separate error handling from the normal program flow.
- The exception should be handled without complex return statement hierarchies
- “Out-of-band” mechanism

C++ Exception Handling

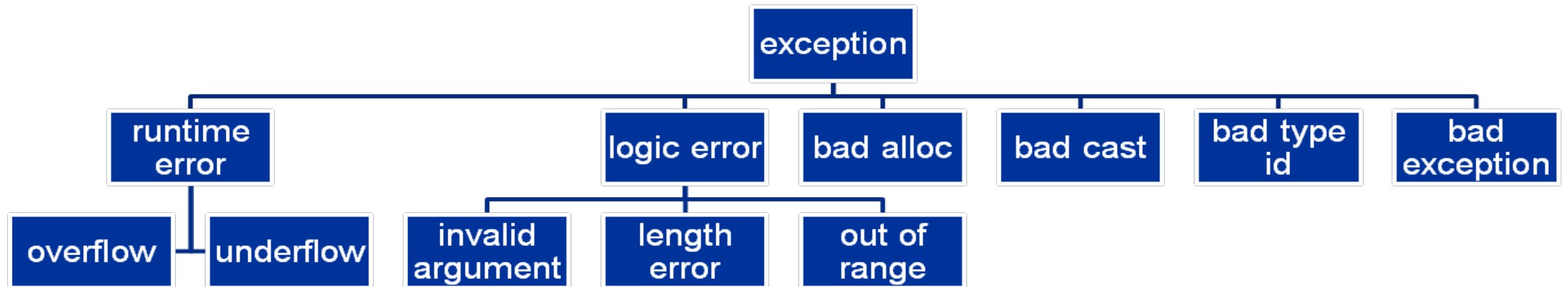


C++ Exception handling consist of 4 parts

- **C++ exception handling consists of 4 parts:**
 - A class (or constant value) representing the exception
 - A try clause in which the exception may occur (typically containing the call to lower level functions)
 - A catch clause for handling the exception
 - And the routine throwing the exception

Live Coding

Standard Library Exception Hierarchy



Using STL exceptions

- **Exceptions in the STL are considered an option**
 - In many places not enabled/used by default
 - Usage can be enabled by setting properties
 - Example `std::ostream` (https://en.cppreference.com/w/cpp/io/basic_ios/exceptions):

```
fstream myData("data1.txt");  
myData.exceptions ( std::ifstream::failbit | std::ifstream::badbit );
```
 - Causes exception `std::ios_base::failure` to be thrown on the respective error (see https://en.cppreference.com/w/cpp/io/ios_base/failure)